



ITL351: Computer Networks Lab
Semester Project Submission 1
Network Simulator
Implementing Physical and Data Link Layers.

Rozain Shakeel
2023BITE040

March 25, 2026

Contents

1	Objective	2
1.1	System Overview	2
2	System Architecture	3
2.1	Layered Architecture in the Simulator	3
2.2	Data Flow Between Layers	4
3	Class Design	5
4	Protocols and Test Cases	8
5	Results, Limitations & Conclusions	10
5.1	Results and Observations	10
5.2	Limitations	10
5.3	Conclusion	10
5.4	Examples	10

Chapter 1

Objective

The objective of this project is to design and implement a network simulator that models the functionality of the Physical Layer and Data Link Layer of the TCP/IP model.

The simulator aims to replicate real-world network behavior including data transmission, collision handling, error detection and correction, and flow control mechanisms. The project also demonstrates how different network devices such as hubs, switches, and bridges operate within a network topology.

1.1 System Overview

This project implements a network simulator in Python that models communication between end devices through various network topologies. The simulator supports both Physical Layer and Data Link Layer functionalities.

At the Physical Layer, data is converted into binary signals and transmitted across the network. The simulator models signal propagation, broadcasting, and collision scenarios.

At the Data Link Layer, data is encapsulated into frames with MAC addressing. The simulator implements error detection and correction using Hamming Code, access control using CSMA/CD, and flow control using the Go-Back-N protocol.

The simulator supports multiple network topologies including point-to-point, star topology using a hub, switch-based networks, and bridge-connected segments. It also provides test cases to validate each functionality.

Chapter 2

System Architecture

The simulator follows a layered architecture inspired by the TCP/IP model, specifically implementing the Physical Layer and Data Link Layer.

The design separates responsibilities between layers to mimic real-world network communication. Each layer performs a specific function and interacts with adjacent layers in a structured manner.

2.1 Layered Architecture in the Simulator

1 Physical Layer (PHY)

The Physical Layer is responsible for transmitting raw bits over the communication medium. In this simulator, it is represented by the Signal class.

Key responsibilities:

- Conversion of data into binary format (bit encoding)
- Transmission of signals between devices
- Simulation of noise and collisions by corrupting signals

2 Data Link Layer (DLL)

The Data Link Layer ensures reliable communication between devices using frames. It is represented by the Frame class and implemented within EndDevice, Switch, and Bridge.

Key responsibilities:

- Framing (encapsulation of data into frames)
- MAC addressing (source and destination identification)
- Error detection and correction using Hamming Code
- Flow control using Go-Back-N protocol
- Access control using CSMA/CD protocol

3 Network Devices

The simulator includes both Physical and Data Link layer devices:

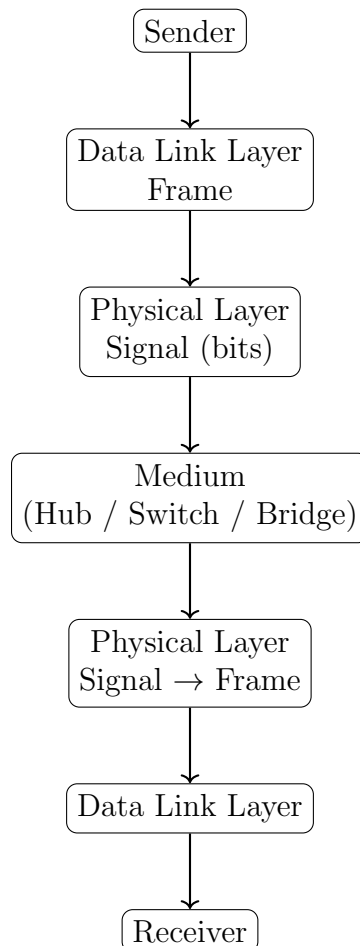
- Hub (Physical Layer Device): Broadcasts incoming signals to all connected devices and may cause collisions.
- Switch (Data Link Layer Device): Uses MAC address learning to forward frames intelligently.
- Bridge (Data Link Layer Device): Connects two network segments and filters traffic.

2.2 Data Flow Between Layers

When a message is sent:

- 1 The Data Link Layer creates a frame with MAC addresses and applies error control.
- 2 The frame is passed to the Physical Layer.
- 3 The Physical Layer converts the frame into a binary signal.
- 4 The signal is transmitted through the medium (hub/switch/bridge).
- 5 At the receiver, the process is reversed:
 - Signal is converted back to frame
 - Frame is checked for errors
 - Data is delivered to the application

This layered approach ensures modularity, making the simulator easier to extend and maintain.



Chapter 3

Class Design

The simulator is designed using an object-oriented approach where each class represents a component of the network or a protocol layer. This modular design improves readability, maintainability, and scalability.

1 **Signal Class (Physical Layer)**

The Signal class represents the transmission of raw data in the form of binary bits over the communication medium.

Responsibilities:

- Encodes messages into binary format
- Carries sender and destination information
- Simulates signal corruption during collisions

This class models the functionality of the Physical Layer by handling low-level data representation and transmission.

2 **Frame Class (Data Link Layer)**

The Frame class represents structured data units used at the Data Link Layer.

Responsibilities:

- Encapsulates data with source and destination MAC addresses
- Converts data into binary format for transmission
- Implements error detection and correction using Hamming Code
- Supports sequence numbering for flow control

This class ensures reliable communication by detecting and correcting transmission errors.

3 **EndDevice Class**

The EndDevice class represents a network node capable of sending and receiving data.

Responsibilities:

- Generates signals for transmission (Physical Layer)
- Creates and processes frames (Data Link Layer)
- Implements Go-Back-N protocol for flow control

- Handles message sending and receiving operations

This class integrates both Physical and Data Link layer functionalities, acting as a complete network host.

4 **Hub Class (Physical Layer Device)**

The Hub class represents a basic networking device operating at the Physical Layer.

Responsibilities:

- Broadcasts incoming signals to all connected devices
- Does not perform any filtering or learning
- Simulates collisions when multiple devices transmit simultaneously

This models the behavior of a real-world hub where all devices share the same communication medium.

5 **Switch Class (Data Link Layer Device)**

The Switch class represents an intelligent networking device operating at the Data Link Layer.

Responsibilities:

- Maintains a MAC address table for forwarding decisions
- Learns source MAC addresses dynamically
- Forwards frames only to the intended destination
- Performs error checking using Hamming Code
- Implements CSMA/CD for access control

This improves network efficiency by reducing unnecessary traffic.

6 **Bridge Class (Data Link Layer Device)**

The Bridge class connects two separate network segments and operates at the Data Link Layer.

Responsibilities:

- Maintains a MAC table mapping devices to segments
- Filters frames if source and destination are on the same segment
- Forwards frames between segments when required
- Reduces network congestion

This helps in segmenting the network and improving performance.

7 **Topology Class**

The Topology class acts as the controller of the entire network.

Responsibilities:

- Creates and manages devices (end devices, hubs, switches, bridges)

- Establishes connections between components
- Builds different network topologies
- Executes simulations and test cases

This class provides an interface for interacting with the network simulator.

Chapter 4

Protocols and Test Cases

The simulator implements key Data Link Layer protocols along with test cases to demonstrate their functionality.

1 Error Control: Hamming Code

Hamming Code is used for error detection and correction in transmitted frames. Each byte of data is encoded with parity bits before transmission.

At the receiver side:

- Errors are detected using parity checks
- Single-bit errors are corrected automatically

Test Case: During transmission through the switch, frames are checked for errors and corrected before delivery.

2 Access Control: CSMA/CD

Carrier Sense Multiple Access with Collision Detection (CSMA/CD) is implemented to manage access to the shared medium.

Working:

- A device senses the channel before transmitting
- If the channel is busy, it waits
- If two devices transmit simultaneously, a collision is detected
- A jam signal is simulated and devices back off before retrying

Test Case: Two devices are made to transmit simultaneously to demonstrate collision detection and backoff behavior.

3 Flow Control: Go-Back-N Protocol

The Go-Back-N protocol is implemented to ensure reliable data transmission using a sliding window mechanism.

Working:

- Multiple frames are sent within a window
- If a frame is lost, all subsequent frames are retransmitted

- Receiver accepts only in-order frames and sends acknowledgments

Test Case: A sequence of frames is sent with simulated loss. The sender retransmits frames after timeout, ensuring correct delivery.

4 Network Topologies Tested

The simulator supports multiple topologies:

- Point-to-Point:

Direct communication between two devices.

- Star Topology (Hub):

All devices are connected to a hub. Data is broadcast, and collisions may occur.

- Switch-Based Network:

Devices communicate through a switch which learns MAC addresses and forwards frames efficiently.

- Bridge Network:

Two segments are connected using a bridge which filters and forwards frames based on MAC addresses.

These test cases validate the correct working of both Physical and Data Link layer functionalities.

Chapter 5

Results, Limitations & Conclusions

The simulator successfully demonstrates the working of Physical and Data Link layers through various network scenarios.

5.1 Results and Observations

- In a hub-based network, signals are broadcast to all devices, leading to collisions when multiple devices transmit simultaneously.
- In a switch-based network, MAC address learning allows efficient frame forwarding, reducing unnecessary traffic.
- The bridge filters traffic between segments, improving network performance.
- Hamming Code successfully detects and corrects errors in transmitted frames.
- Go-Back-N protocol ensures reliable transmission by retransmitting lost frames.

5.2 Limitations

- The simulator does not include real-time delays or actual network timing.
- Acknowledgments in Go-Back-N are simulated rather than transmitted as real frames.
- The system does not include a graphical interface and operates through console output.
- Error simulation is limited and does not include complex noise models.

5.3 Conclusion

The project successfully implements a network simulator that models key functionalities of the Physical and Data Link layers. It demonstrates important networking concepts such as framing, error control, flow control, and access control. The modular design allows easy extension for future improvements, including additional protocols and layers.

5.4 Examples

```

(base) rozainshakeel@Macbook-Air Programming_assignment1_CN % python3 CN1.py

=====
Network Simulator – Protocol Stack
=====

Select topology:
  1. Point-to-point
  2. Star (Hub)
  3. Switch Star
  4. Bridge

Enter choice: 1
Enter name of device 1: A
Enter name of device 2: B
EndDevice 'A' created
EndDevice 'B' created
Direct connection: A ↔ B

Point-to-point topology built – A ↔ B

=====
Topology : Point-to-Point
Devices  : A, B
Hubs     : None
Switches :
Bridges  :
=====

What do you want to do?
  1. Send a message
  6. Exit

Enter choice: 1
Who is sending? B
Enter message: hello
Enter destination (or press Enter for broadcast): A
B wants to send to A
Signal Generated:
From : B
To   : A
Message : hello
Bits : 01101000 01100101 01101100 01101100 01101111

A received:
From : B
To   : A
Message : hello
Bits : 01101000 01100101 01101100 01101100 01101111

What do you want to do?
  1. Send a message
  6. Exit

Enter choice: 6

Exiting simulator. Goodbye!

```

Figure 5.1: Point to point(message send)

```

(base) rozainshakeel@Macbook-Air Programming_assignment1_CN % python3 CN1.py

=====
Network Simulator – Protocol Stack
=====

Select topology:
  1. Point-to-point
  2. Star (Hub)
  3. Switch Star
  4. Bridge

Enter choice: 1
Enter name of device 1: A
Enter name of device 2: B
EndDevice 'A' created
EndDevice 'B' created
Direct connection: A ↔ B

Point-to-point topology built – A ↔ B

=====
Topology : Point-to-Point
Devices  : A, B
Hubs     : None
Switches :
Bridges  :
=====

What do you want to do?
  1. Send a message
  6. Exit

Enter choice: 1
Who is sending? B
Enter message: hello
Enter destination (or press Enter for broadcast): A
B wants to send to A
Signal Generated:
From : B
To   : A
Message : hello
Bits : 01101000 01100101 01101100 01101100 01101111

A received:
From : B
To   : A
Message : hello
Bits : 01101000 01100101 01101100 01101100 01101111

What do you want to do?
  1. Send a message
  6. Exit

Enter choice: 6

Exiting simulator. Goodbye!

```

Figure 5.2: switch (message send)

```

(base) rozainshakeel@Macbook-Air Programming_assignment1_CN % python3 CN1.py

=====
Network Simulator – Protocol Stack
=====

Select topology:
  1. Point-to-point
  2. Star (Hub)
  3. Switch Star
  4. Bridge

Enter choice: 1
Enter name of device 1: A
Enter name of device 2: B
EndDevice 'A' created
EndDevice 'B' created
Direct connection: A ↔ B

Point-to-point topology built – A ↔ B

=====
Topology : Point-to-Point
Devices  : A, B
Hubs     : None
Switches :
Bridges  :
=====

What do you want to do?
  1. Send a message
  6. Exit

Enter choice: 1
Who is sending? B
Enter message: hello
Enter destination (or press Enter for broadcast): A
B wants to send to A
Signal Generated:
From : B
To   : A
Message : hello
Bits : 01101000 01100101 01101100 01101100 01101111

A received:
From : B
To   : A
Message : hello
Bits : 01101000 01100101 01101100 01101100 01101111

What do you want to do?
  1. Send a message
  6. Exit

Enter choice: 6

Exiting simulator. Goodbye!

```

Figure 5.3: switch (collision)

```

(base) rozainshakeel@Macbook-Air Programming_assignment1_CN % python3 CN1.py

What do you want to do?
1. Send a message
3. Go Back N transmission
4. Show MAC table
5. Simulate CSMA/CD collision
6. Exit

Enter choice: 3
Who is sending? c
Enter destination: e
How many frames? 1
  Frame 0: hello
Window size: 3
Drop probability (0.0 to 1.0): 0.01

c starting Go-Back-N transmission
Window size: 3 | Drop probability: 0.01
Total frames to send: 1

Sending frame 0: 'hello'
Src MAC  : C8:E3:D2:66:03:EB
Dst MAC  : 9A:0D:DD:F3:65:8C
Seq No   : 0
Data     : hello
Bits     : 01101000 01100101 01101100 01101100 01101111
Hamming  : ['010011011000', '100111000101', '000011001100', '000011001100', '110111001111']

[PHY] Received frame from DLL layer
[PHY] Frame converted to signal
[PHY] Bits: 01000011 00111000 00111010 01000101 0011...

[DLL] switch received signal from c

[PHY] Signal received – converting to frame for DLL layer
[PHY] Bits reconstructed into frame – passing to DLL layer
[DLL] Hamming check passed – no errors
[DLL] Forwarding to e

[PHY] Signal received – converting to frame for DLL layer
[PHY] Bits reconstructed into frame – passing to DLL layer

e received frame:
Src MAC  : C8:E3:D2:66:03:EB
Dst MAC  : 9A:0D:DD:F3:65:8C
Seq No   : 0
Data     : hello
Bits     : 01101000 01100101 01101100 01101100 01101111
Hamming  : ['010011011000', '100111000101', '000011001100', '000011001100', '110111001111']

e accepted frame – Data: 'hello'

[GBN] Receiver sending ACK 1 (frames up to 0 received ✓)

ACK received for frames up to 0

All frames acknowledged – transmission complete!

```

Figure 5.4: Go Back N